cocotb & verilator 效率对比实验

1. 实验测试样例

使用 corex/my_c_test/src/binary_search.c 进行测试，分别使用cocotb+verilator和纯verilator进行测试，以测试其仿真效率。同时对verilator增加RTL coverage进行实验，以测试增加覆盖率测试对仿真效率的影响。

- cocotb脚本位于 corex/my_c_test/testcases/test_core_binary_search.py
- 纯verilator的脚本位于 corex/my_c_test/tb_verilator/sim_main.cpp

使用方式和实验结果见下：

2. cocotb + verilator 仿真效率

```
cd corex/my_c_test
make clean
make run TEST=binary_search
```

参数：

- REAL TIME:外部python解释器启动整个测试框架的总耗时
- Test Wall Time:仿真用例执行耗时，只包含测试用例进入执行阶段到测试结束的时间
- Pure Sim Dynamic Time:纯仿真时间

修改轮次：corex/my_c_test/testcases/test_core_binary_search.py 修改23行 RUNS = int(os.getenv("RUNS", "50"))即可修改总轮次

3. 纯verilator仿真效率

```
cd corex/my_c_test/tb_verilator
make clean-all
```

参数：

- Total process Wall TIME：端到端总运行时间
- Setup/Init Overhead：初始化/环境开销
- Pure Sim Time：纯RTL仿真耗时可通过计算得出：Total process Wall TIME - Setup/Init Overhead
- Pure Sim Throughput：纯仿真吞吐率，指扣除初始化开销后，纯粹衡量 RTL 逻辑运算的速度。
- Wall-Clock Throughput：端到端综合吞吐率，指把初始化开销也算上后，整体程序摊分到每个 Cycle 的平均速度。

4. 实验数据对比

测试方案 (Scheme)	Coverage	运行轮次	仿真周期数 (Cycles)	纯仿真耗时 (Pure Sim Time)	仿真真实耗时 (Real Time)	纯仿真单周期耗时 (ns/cycle)	纯仿真吞吐率 (Pure Throughput)
`cocotb_run_1`	×	1	282	5.550 ms	**0.08s**	19,679.2 ns	**50.82 Kcycles/s**
`cocotb_run_50`	×	50	14,100	266.322 ms	**0.35s**	18,888.1 ns	**52.94 Kcycles/s**
`verilator_run_1`	×	1	280	0.098 ms	**1.233 ms**	350.2 ns	**2,855.60 Kcycles/s** (2.86 M)
`verilator_run_50`	×	50	14,000	4.043 ms	**4.780 ms**	288.6 ns	**3,464.64 Kcycles/s** (3.46 M)
`verilator_run_1_coverage`	*V*	1	280	1.406 ms	**24.739 ms**	4,939.2 ns	**202.46 Kcycles/s**
`verilator_run_50_coverage`	*V*	50	14,000	66.349 ms	**90.974 ms**	4,739.0 ns	**211.02 Kcycles/s**